1/8/26
Saturday

CSA1407 - Compiler Design
Test

R. Balaji
192324255
B.Tech - AI-DS

64

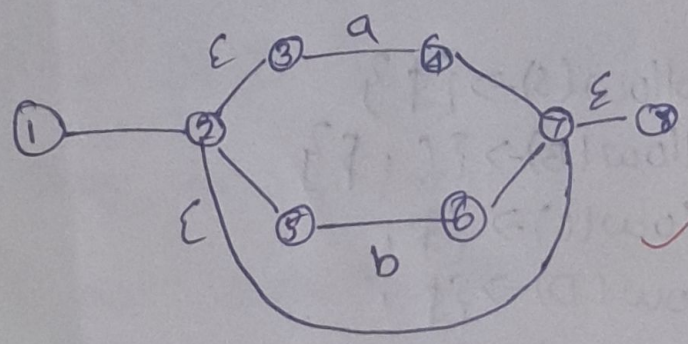
① $E \rightarrow E+E \mid id$

Soln:

ambigious → $E \rightarrow E+E$, $E \rightarrow id$, it is ambigous beaus Both left and right has same ....

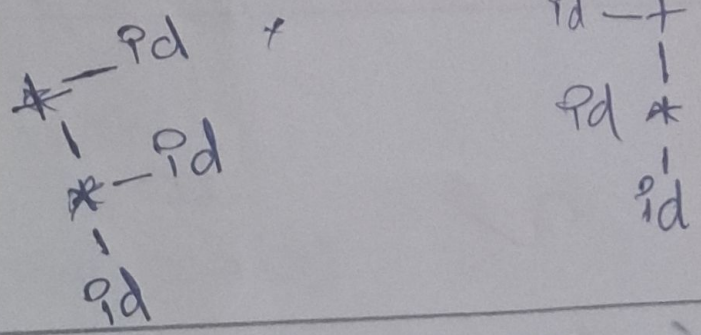
② NFA → $a(a|b)^*a$

Soln!



③ Parse tree

Soln: → id + id * id



④ Eliminate left recursion from the grammer $A \rightarrow A\alpha \mid \beta$

$A \rightarrow A\alpha \mid \beta \Rightarrow A \rightarrow A\alpha$, $A \rightarrow \beta \Rightarrow A \rightarrow \beta A'$, $A' \rightarrow \alpha A' / \varepsilon$

$A \rightarrow \beta A'$

$A' \rightarrow \alpha A' / \varepsilon$

⑤ S → ifEts | ifEtS es | a

Soln:

S → ifEts | ifEts es | a

S → ifEts (i) / ifEts es | ε
α β α γ

S → ifEts S'

S' → es | ε

formula:

A → αβ | αγ | ε

A → αA'

A' → β | γ

2

⑥ First and Follow

S → aBDh
B → cC
C → b C | ε
D → EF
E → g | ε
F → f | ε

First(S) → {a}
First(B) → {c}
First(C) → {b, ε}
First(D) → {g, ε}
First(E) → {g, ε}
First(F) → {f, ε}

Follow(S) → {$}
Follow(B) → {(, $}
Follow(C) → {$,
Follow(D) → {$,
Follow(E) → {$,
Follow(F) → {$,

αBβ

3

⑦ S → sa|sb|cd.

soln: S → sa|sb|cd

S → sa
S → sb
S → cd
S → bs'
S → as'

∴ S → as' | bs' | ε

The gramar is left recursion.

2

⑧ E → TE', E' → +TE' | ε

↳ First(E') = {+, ε}
FIRST

soln:

2

3

(9) $S \to (S)S \mid \epsilon$, check (()) is valid

Soln:

$S \to (S)S$

$S \to \epsilon$

S implies (), so (()) is valid.

(10) what action are performed on shift reduce parsing.

Soln:

(1) if it implies the similar word of $\to E$, have to shift the word to left / cancel it.

(2) if it is in order (id, (,), +, - / E) we have to shift that to left / next.

(3) if not reduce the item / make to move next.

E * id * id reduce

(11) Find closure:

$S' \to S$

$S \to CC$

$C \to cC \mid d$

Item: [S' → .S]

Soln:

Goto $(I_0, S) \equiv I_1$

$S' \to S.$ ✓

Goto $(I_0, C) \equiv I_2$

$S \to .CC$

$C \to .cC$

$C \to .d$

$C \to d.$ ✓

Goto $(I_0, S) \equiv I_3$

$S \to C.C$

$C \to C.cC$

Goto $(I_0, S) \equiv I_3$

$S \to CC.$ ✓

Goto $(I_0, C) \equiv I_4$

$C \to cC.$ ✓

[$S' \to S.$] ✓

[$S \to CC.$] ✓

[$C \to CC.$] ✓ [$C \to d.$]

12) FIRST, S → AB | A → a | B → b

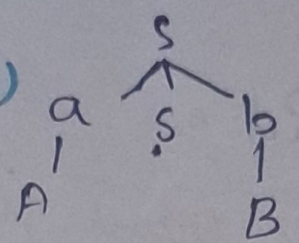
S → aSb | ab , check aabb is accepted.

Soln:

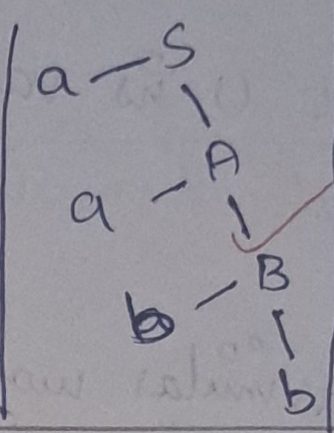
S → AB

A → a, B → b

(ii)



aabb is a accepted



(i)

FIRST(S) = {A}

FIRST(A) = {a}

FIRST(B) = {b}

3

13) FIRST and Follow → A → A'd , dA' ∝ BB

S → A

A → aB | Ad

B → b

C → g

2

FIRST(S) → {a}

FIRST(A) → {a, d}

FIRST(B) → {b}

FIRST(C) → {g}

Follow(S) → {$}

Follow(A) → {$, b}

Follow(B) → {$, b}

Follow(C) → {$, g, b}

14) Simple code for Addition (4ex code)

Code:

LDF R1 , R2

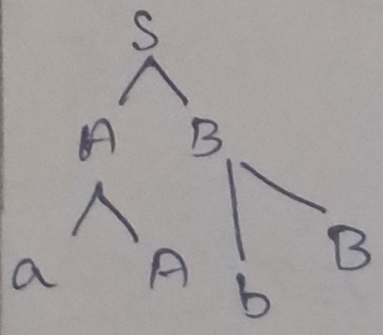
ADF R1 , R1 , R2 //

STF id1 , R1

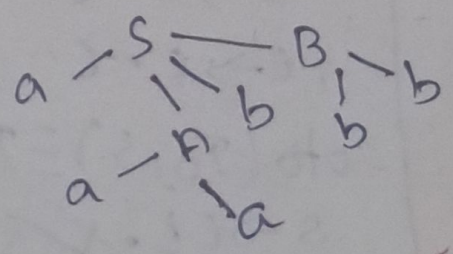
1

15) LMD and RMD:

S → AB , A → aA | a , B → bB | b



LMD



RMD

2

⑥ S → AB
A → aA | ε
B → bB | c.

αBβ

AB		
aA	ε	
bB	c	
αBB	αBβ	

Soln:

Follow (S) → {$, ●}
Follow (A) → {$, a}
Follow (B) → {$, b, c}

17) Rule to Eliminate left recursion and left factoring.

19) Stack implementation of shift reduce parsing.

Stack | String | operation

(Y) Pd * Pd * id

FIRST(E) = {(, Pd}
FIRST(E') = {ε, Pd}
FIRST(T') = {*, ε}
FIRST(T) = {(, Pd}
FIRST(F) = {(, Pd}

Follow(E) = {$}
Follow(E') = {$,)}
Follow(T) = {+, $,)}
Follow(T') = {+, $,)}
Follow(F) = {+, *, $}

Norm	Pd	+	*	(	)	$
E	E → TE'			E → TE'		
E'		E' → +TE'/ε				
T	T → FT'			T → FT'		
T'		T' → ε	T' → *FT'/ε		T' → ε	T' → ε
F	F → (E)/Pd			F → (E)/Pd		

(14) Shift reduce:

$E \to TE'$

$E' \to +TE' | \varepsilon$

$T \to FT'$

$T' \to *FT' | \varepsilon$

$F \to (E) | id$

id + id * id

Recursive

Grammar	shift string	shift
$ E 4	id + id * id $	$E \to TE'$
$ T' E	id + id * id $	$T \to FT'$
$F \to id$	id + id * id $	$F \to id$ (E)
$E \to E'T$	+ id * id $	$E \to TE'$
~~$T \to T * F$~~ $E \to TE'$	* id * id $	$T \to FE'$
$E' \to +TE'$	id * id $	$E' \to +TE'$
$T \to T'F$	id * id $	$T \to FT'$
$F \to id T'$	* id $	$F \to (E)$
$T' \to FT'$	* id $	$T' \to *FT'$
$F \to id FT'$	id $	$T' \to *FT' \| \varepsilon$
$T \to FT'F$	id $	$T \to FT'$
$F \to id$	$	$F \to (E) \| id$

Shift reduce:

$E \rightarrow E+E$

$E \rightarrow E*E$

$E \rightarrow (E)$

$E \rightarrow Id$

Stack	input	shift/reduce.
id_1	$id_1 + id_2 * id_3 \$$	Shift
$E+$	$id_2 * id_3 \$$	Shift
$E+id_2$	$* id_3 \$$	reduce
$E+E$	$* id_3 \$$	reduce
E	$* id_3 \$$	reduce
$E*$	$id_3 \$$	Shift
$E*id_3$	$\$$	reduce
$E*E$	$\$$	Reduce
E	$\$$	reduce.
		Accepted

7

20)
```
#include <stdio.h>
int main(){
  int a=10, b=20;
  int sum=a+b;
  printf("%d",sum);
  return 0;
}
```

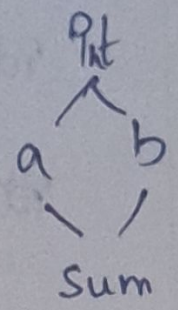
<id,><=><id2><+><id3>

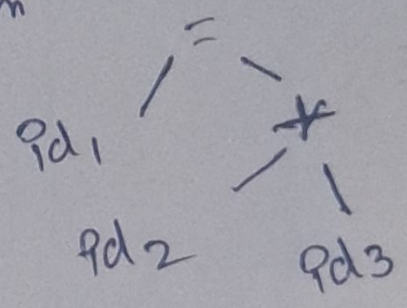
<int><a><=><10><,><b><=><20>;

Soln:

a) Lexical Analysis
↓
Syntax Analysis
↓
Semantic Analysis
↓
Intermediate code Generation
↓
Code Optimization
↓
Code Generator.

c)





parse tree

↳ <id1> = <id2> <+> <id3>

b)

Sum=a+b
t1 = id3
t2 = t1 + id2
t3 = t2
id1 = t3

⇒ code optimization

↳ t1 = id2 + id3
id1 = t1